

Cel eksperymentu

Ogólna idea

Celem eksperymentu jest sprawdzenie, jak topologia połączeń między wyspami oraz strategia wyboru migrantów wpływają na skuteczność asynchronicznego wyspowego algorytmu ewolucyjnego.

Badamy nie tylko to, jaki wynik algorytm osiąga na końcu, ale też jak szybko do tego wyniku dochodzi. Interesuje nas więc zarówno jakość finalnego rozwiązania, jak i tempo zbiegania w trakcie wykorzystania budżetu ewaluacji.

W praktyce chcemy móc odpowiedzieć na pytania:

- czy jedna topologia regularnie daje lepsze wyniki od innej,
- czy dana strategia migracji jest lepsza dla konkretnych funkcji celu,
- czy strategię migracji różni się tempem zbiegania,
- czy zwiększenie liczby wysp zmienia przewagę danej topologii albo strategii,
- czy wyniki dla funkcji ciągłych i dyskretnych mają podobny charakter.

Model wyspowy

W eksperymencie każda wyspa prowadzi własną ewolucję populacji. Co pewien interwał migracji wyspy wymieniają wybrane osobniki z innymi wyspami zgodnie z zadaną topologią. Topologia mówi, które wyspy mogą przekazywać sobie migrantów, a strategia migracji mówi, jakie osobniki są wybierane do migracji.

To oznacza, że badamy dwa główne mechanizmy:

- strukturę komunikacji między wyspami,
- sposób wyboru informacji genetycznej przekazywanej między wyspami.

Przestrzeń przeszukiwania

Eksperyment obejmuje dwie klasy przestrzeni przeszukiwania.

Problemy ciągłe

Dla problemów ciągłych osobnik jest wektorem liczb rzeczywistych. Lokalna wersja eksperymentu używa zmniejszonego wymiaru, żeby dało się policzyć macierz na komputerze lokalnym:

```
liczba zmiennych: 30
budżet ewaluacji: 1000
populacja: 16
offspring: 4
```

Główne funkcje ciągłe:

- **Sphere** - funkcja gładka i unimodalna; służy jako bazowy test szybkiej zbieżności.

- **Rastrigin** - funkcja wielomodalna z wieloma minimami lokalnymi; sprawdza, czy migracja pomaga unikać utknięcia w lokalnych strukturach.
- **Ackley** - funkcja wielomodalna z charakterystycznym płaskim obszarem i globalnym minimum; sprawdza odporność strategii na trudniejszy krajobraz.

Problemy dyskretne

Dla problemów dyskretnych osobnik jest reprezentowany binarnie. Lokalna wersja eksperymentu używa:

```
liczba bitów: 60
budżet ewaluacji: 1000
populacja: 16
offspring: 4
```

Główne problemy dyskretne:

- **labs_binary** - Low Autocorrelation Binary Sequence; trudny problem binarny oparty na autokorelacji sekwencji.
- **trap5** - zwodnicza funkcja trap o blokach 5-bitowych; sprawdza, czy migracja rozprasza dobre bloki czy wzmacnia mylące lokalne struktury.
- **nk_k4** - krajobraz NK z lokalnymi zależnościami między bitami; testuje wpływ epistazy i struktury topologii.

Badane topologie

Topologia określa, z którymi wyspami dana wyspa może się komunikować.

Topologie regularne:

- **ring** - lokalne sąsiedztwo; informacja rozchodzi się wolniej.
- **torus** - siatka z zawijaniem; pośredni poziom lokalności i łączności.
- **complete** - każda wyspa może komunikować się z każdą; informacja rozchodzi się bardzo szybko.

Topologie losowe:

- **rt_er_d4_s1** - graf Erdos-Renyi o oczekiwanym stopniu około 4.
- **rt_er_d8_s1** - graf Erdos-Renyi o oczekiwanym stopniu około 8.
- **rt_ws_k4_p010_s1** - graf Watts-Strogatz z lokalnymi połączeniami i niewielkim prawdopodobieństwem przełączeń.

Topologie losowe są zapisane jako deterministyczne pliki JSON, żeby każde powtórzenie używało tej samej struktury grafu.

Strategie wyboru migrantów

Porównujemy cztery strategie wyboru osobników wysyłanych z wyspy:

- **random** - losowy wybór migrantów.
- **best** - wybór najlepszych osobników.

- **worst** - wybór najgorszych osobników.
- **maxDistance** - wybór osobników maksymalnie odległych, czyli strategia nastawiona na różnorodność.

Chcemy sprawdzić, czy bardziej eksploatacyjne strategie (**best**) albo bardziej dywersyfikujące strategie (**random**, **maxDistance**) lepiej działają w zależności od funkcji celu i topologii.

Lokalna macierz eksperymentu

Lokalna wersja eksperymentu jest przeskalowana względem pierwotnej macierzy HPC. Ma zachować ten sam sens badawczy, ale być policzalna na komputerze lokalnym.

Dla każdej z czterech grup:

```
3 funkcje/problemy
x 3 topologie
x 3 liczby wysp
x 4 strategie migracji
x 3 powtórzenia
= 324 runy
```

Grupy:

- ciągłe funkcje + topologie regularne,
- ciągłe funkcje + topologie losowe,
- dyskretne funkcje + topologie regularne,
- dyskretne funkcje + topologie losowe.

Liczby wysp:

```
12, 24, 36
```

Każdy mini-batch obejmuje jedną kombinację:

```
funkcja/problem + topologia + liczba wysp
```

i dla niej odpala:

```
4 strategie migracji x 3 powtórzenia = 12 runów
```

Główne metryki

Eksperyment ma dwa główne typy metryk.

Jakość końcowa

Najważniejsze metryki jakości końcowej:

- `best_final_mean` - średni najlepszy wynik końcowy z powtórzeń.
- `mean_final_mean` - średni wynik końcowy po wyspach i powtórzeniach.
- `best_final_std` - zmienność najlepszego wyniku między powtórzeniami.

Dla badanych problemów niższa wartość fitness oznacza lepszy wynik.

Tempo zbiegania

Najważniejsze metryki szybkości:

- `eval_to_50pct_improvement_mean` - średnia liczba ewaluacji potrzebna do osiągnięcia 50% całkowitej poprawy danego runu.
- `eval_to_90pct_improvement_mean` - średnia liczba ewaluacji potrzebna do osiągnięcia 90% całkowitej poprawy.
- `best_at_25pct_budget_mean`, `best_at_50pct_budget_mean`, `best_at_75pct_budget_mean`, `best_at_100pct_budget_mean` - jak dobry wynik był dostępny po wykorzystaniu określonej części budżetu.

Dla metryk `eval_to_*` niższa wartość oznacza szybszą zbieżność.

Oczekiwany efekt analizy

Po policzeniu batchy chcemy otrzymać tabele pozwalające powiedzieć np.:

- dla danej funkcji celu topologia `ring/torus/complete` regularnie daje lepsze wyniki,
- strategia `best` szybciej zbiega, ale nie zawsze daje najlepszy wynik końcowy,
- strategia `maxDistance` może pomagać w funkcjach wielomodalnych, ale nie musi być najlepsza na gładkiej funkcji `Sphere`,
- przy większej liczbie wysp przewaga danej topologii może się zmieniać,
- problemy dyskretne mogą preferować inne strategie migracji niż problemy ciągłe.

Pliki wynikowe do wnioskowania

Najważniejsze pliki do analizy po agregacji:

```
local_matrix_computation/analysis/combined_runs.csv
local_matrix_computation/analysis/comparison_groups.csv
local_matrix_computation/analysis/strategy_ranking.csv
local_matrix_computation/analysis/strategy_convergence_ranking.csv
local_matrix_computation/analysis/topology_ranking.csv
local_matrix_computation/analysis/topology_convergence_ranking.csv
```

`combined_runs.csv` zawiera pojedyncze runy, a `comparison_groups.csv` zawiera agregaty po powtórzeniach. Rankingi strategii i topologii są pomocnicze: pozwalają szybko znaleźć najlepsze warianty, ale finalne wnioski powinny brać pod uwagę także liczbę powtórzeń i odchylenie standardowe.

Najważniejszy warunek poprawnych wniosków

Do wnioskowania potrzebne są kompletne powtórzenia. Pojedynczy smoke run sprawdza tylko, czy pipeline działa. Sensowne porównania zaczynają się wtedy, gdy dla danej kombinacji mamy:

```
run_count = 3
```

czyli trzy niezależne powtórzenia.